

LIVELLO TRASPORTO

Il livello trasporto crea un collegamento logico tra due applicazioni su host diversi.

In sostanza controlla e organizza l'invio dei dati, ma la trasmissione fisica è compito del livello rete e inferiori.

POSIZIONE (iso-osi): tra livello rete e livello sessione

Confronto con il Livello Data Link

Il livello trasporto svolge **funzioni simili** al livello Data Link, come:

- Controllo degli errori
- Mantenimento della sequenza corretta dei dati
- Controllo di flusso

Differenza fondamentale:

- Il **Data Link** opera sul **canale di trasmissione fisico** (connessione tra due nodi adiacenti).
- Il **Trasporto** opera sulla **connessione logica end-to-end** (tra due applicazioni su host remoti, anche se molto distanti).

I pacchetti di dati si chiamano **TPDU** (Transport Protocol Data Unit)

Indirizzo livello 4: Indirizzo IP + numero di porta → **SOCKET** (es. 192.168.1.10 : 80)

La **SOCKET** identifica un processo applicativo in esecuzione

Indirizzo IP → Identifica l'host

Numero di porta → Identifica l'applicazione nell'host (**numero intero a 16 bit**)

MULTIPLEXING/DEMULTIPLEXING

PROBLEMA:

In un host possono esserci più applicazioni attive contemporaneamente.

SOLUZIONE:

- In host mittente: MULTIPLEXING
Il livello trasporto riceve i dati dalle socket e aggiunge informazioni di trasporto (**header**).
- In host destinatario: DEMULTIPLEXING
Il livello trasporto legge l'header e determina a quale socket dare i dati.

TIPI NUMERO DI PORTA

- **FISSO (Well-Known Ports):**
Range: 0 – 1023
Utilizzate per le applicazioni più importanti (HTTP, FTP, SMTP, DNS...).
- **REGISTRATE (Registered Ports):**
Range: 1024 – 49151
Utilizzate per applicazioni e servizi specifici registrati.
- **DINAMICHE (Dynamic Ports):**
Range: 49152 – 65535
Utilizzate principalmente dai client e per processi server temporanei.

PRIMITIVE

Le primitive sono le istruzioni che un programma può dare al servizio trasporto.

1. **LISTEN**: attesa di connessione
2. **T-CONNECT**: apertura della connessione
3. **SEND DATA**: invio dei dati (contenuto)
4. **RECEIVE DATA**: ricezione dei dati
5. **T-DISCONNECT**: chiusura della connessione

Metodi

Per ognuna di queste primitive vengono utilizzati questi metodi:

- **.request()**: chiedo al servizio di fare qualcosa
- **.indication()**: il servizio avverte che è successo qualcosa (evento).
- **.response()**: rispondo all' evento che mi è stato indicato
- **.confirm()**: il servizio segnala l'arrivo di una conferma (la mia richiesta è stata completata con successo).

I protocolli delle applicazioni si appoggiano al livello trasporto, che utilizza due tipi di servizi:

- **TCP (transmission control protocol)** → Orientato alla connessione
- **UDP (user datagram protocol)** → Senza connessione

Tipi di Connessione

1. Connessione Asincrona (Connection-less)

Tipica di **UDP**

- Nessuna conferma di ricezione al mittente
- Più veloce ma inaffidabile
- Flusso: PRIMITIVA.REQUEST (A) → PRIMITIVA.INDICATION (B)

2. Connessione Sincrona (Connection-oriented)

Tipica di **TCP**

- Prevede conferma e risposta
- Flusso: REQUEST (A) → INDICATION (B) → RESPONSE (B) → CONFIRM (A)

ENTITÀ	AZIONE	DESCRIZIONE
A	invia T-CONNECT.request()	A vuole connettersi
B	riceve T-CONNECT.indication()	qualcuno vuole connettersi a B
B	invia T-CONNECT.response()	B accetta la connessione
A	riceve T-CONNECT.confirm()	conferma di connessione stabilita

UDP

Non prevede l'uso di una connessione (connection-less) fra mittente e destinatario

Ogni datagram UDP è **indipendente**.

Servizio **best effort**: Non si ha garanzia sulla consegna dei dati (datagrammi).

Simile a IP, ma ha la capacità di distinguere tra più destinazioni all'interno dello stesso host grazie alle porte (**quindi è in grado di consegnare all'applicazione corretta**)

I datagram possono seguire percorsi diversi e quindi non arrivano in ordine.

Segmento UDP

Ogni datagramma è gestito in modo indipendente e supporta modalità di trasferimento tra processi di tipo:

- punto-punto
- bidirezionale

UDP **supporta il multicast** (un mittente può inviare lo stesso datagramma a più destinatari contemporaneamente).

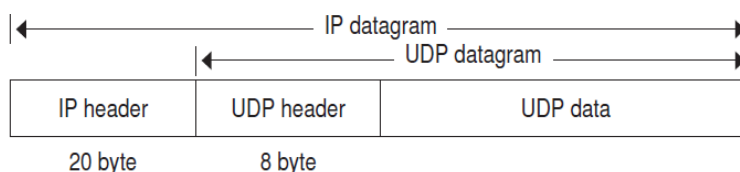
Struttura del datagramma UDP

Il datagramma UDP viene **incapsulato** all'interno di un pacchetto IP.

La sua dimensione **non può superare** la dimensione massima della parte dati del datagramma IP.

È composto da due parti:

- **header** di 8 byte (fisso)
- **payload** (parte dati) di lunghezza variabile (default: 536 byte -> massimo: 65.535 byte)



La dimensione del payload viene decisa dal mittente durante il setup della comunicazione.

Datagramma UDP – header

Il pacchetto UDP è formato da:

- **Numero di porta del mittente** (16 bit)
- **Numero di porta del destinatario** (16 bit)
- **Lunghezza** dei dati del datagramma UDP (in byte, 16 bit)
- **Checksum** per il controllo d'errore (16 bit) – **opzionale**

Il checksum viene calcolato dall'header IP (pseudo intestazione, contenente ip del mittente e destinatario). In questo modo si verifica l'integrità dell'intero pacchetto senza duplicare informazioni già presenti a livello rete.

Trasmissione UDP – esempio pratico

Supponiamo di trasmettere un datagramma UDP contenente la stringa di testo “**pippo**” sulla porta **3001**

Il receiver si mette in attesa (listen) sulla porta 3001.

Quando il sender invia il datagramma sulla porta 3001 del receiver, quest'ultimo riceve e stampa il contenuto.

Conclusione: mittente e destinatario devono aver concordato preventivamente la porta da usare e il mittente deve conoscere l'indirizzo IP del destinatario.

Vantaggi di UDP

- Non richiede di stabilire una connessione → **nessun ritardo** di setup.
- Non mantiene lo stato della connessione → un server può gestire **molte più client** contemporaneamente.
- **Sovraccarico** dovuto all'header **minimo: 8 byte** (contro i 20 byte di TCP).
- Il controllo del livello applicativo è più efficace: il mittente **non viene mai bloccato**.

TCP

TCP è **molto più diffuso** di UDP e rappresenta il protocollo di trasporto più utilizzato su Internet.

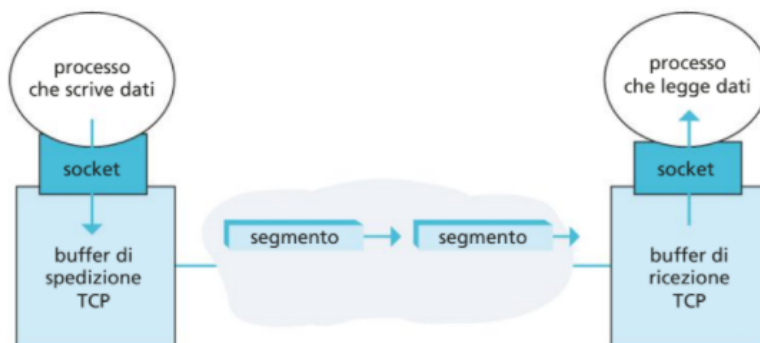
Caratteristiche principali di TCP:

- Servizio **connection-oriented** (orientato alla connessione) e **affidabile**.
- **Riordina** automaticamente i pacchetti in ricezione (i pacchetti seguono percorsi diversi e arrivano disordinati ma vengono ordinati da protocollo)
- Consente applicazioni che richiedono trasferimento di file **senza errori** (web, posta elettronica, FTP, ecc.).
- Effettua un **controllo di congestione end-to-end** che limita il traffico e permette una condivisione equa delle risorse di rete.
- Richiede **molte più risorse** (memoria e CPU) rispetto a UDP.

TCP è un protocollo **connection-oriented**:

- Prima di trasferire i dati deve essere instaurata una connessione tramite una opportuna procedura di segnalazione.
- Le connessioni TCP sono sempre **full-duplex**: esiste un flusso di dati in entrambe le direzioni.
- TCP lavora sempre in combinazione con protocolli a livello rete di tipo datagram (principalmente IP).

Comunicazione TCP – processo applicativo



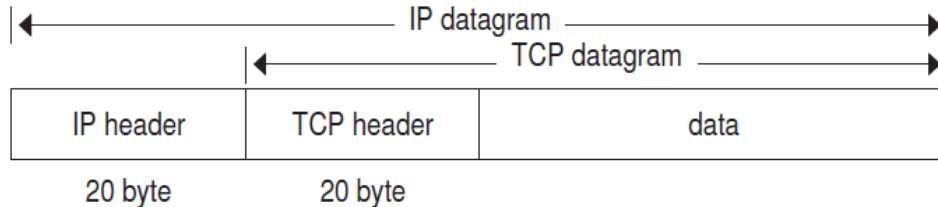
Il processo applicativo deve passare al modulo TCP diverse informazioni per gestire correttamente la trasmissione:

- Source address (network + host + port)
- Destination address (network + host + port)
- Next packet sequence number (numero di sequenza per il prossimo pacchetto)
- Current buffer size (dimensione del buffer del mittente)
- Next write position (posizione nel buffer dove scrivere i nuovi dati)
- Next read position (posizione da cui TCP legge i dati per formare il segmento)
- Timeout/flag (tempo dopo il quale ritrasmettere i dati non confermati)

Il segmento TCP

Il segmento TCP viene incapsulato nel datagramma IP ed è diviso in:

- **Intestazione:** 20 byte
- **Payload:** variabile, fino a 64 KB (contiene i dati dell'applicazione)



Struttura del segmento TCP

Il segmento TCP ha una struttura ben definita con campi fissi e opzionali.

Campi del segmento TCP

- **Source port number** (16 bit) – porta del mittente
- **Destination port number** (16 bit) – porta del destinatario
- **Sequence number** (32 bit) – numero di sequenza del **primo byte** di dati nel segmento
- **Acknowledgment number** (32 bit) – numero di riscontro: indica il prossimo sequence number che il ricevitore si aspetta di ricevere
- **Data offset** (4 bit) – lunghezza dell'header TCP (in parole da 32 bit)
- **Flags** (8 bit) – bit di controllo (SYN, ACK, FIN, RST, PSH, URG, ecc.)
- **Window size** (16 bit) – dimensione della finestra di ricezione (quanti byte il ricevitore è pronto ad accettare)
- **Checksum** (16 bit) – obbligatorio in TCP (calcolato come in UDP)
- **Urgent pointer** (16 bit) – usato solo se il flag URG = 1; indica l'ultimo byte urgente
- **Options** (variabile, fino a 32 bit) – l'opzione più importante è la **MSS** (Maximum Segment Size); valore di default 536 byte
- **Data** – dati provenienti dal livello superiore

Controllo di flusso nel segmento TCP

Il mittente utilizza il valore di **MSS** (Maximum Segment Size) per non sovraccaricare il destinatario.

Inoltre, ogni segmento deve rispettare il **MTU** (Maximum Transmission Unit) delle reti che attraversa.

La connessione TCP - Apertura (Three-Way Handshake)

Prima di poter scambiare dati, client e server devono **sincronizzare** i loro numeri di sequenza e accordarsi sulla connessione. Questo avviene con la famosa **procedura a tre vie** (Three-Way Handshake).

Passo per passo:

1. Passive Open (Server)

Il server avvia l'applicazione e rimane in **attesa passiva** sulla porta specifica (es. porta 80 per un web server).

Il server è pronto ad accettare connessioni, ma non fa nulla fino a quando non arriva una richiesta.

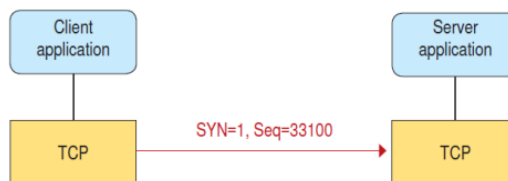
2. Active Open (PT 1)

Quando un client vuole comunicare con un server, invia la richiesta TCP di **Active Open** tramite l'applicativo che conosce l'IP del destinatario e il numero di porta.



3. Active Open (PT 2)

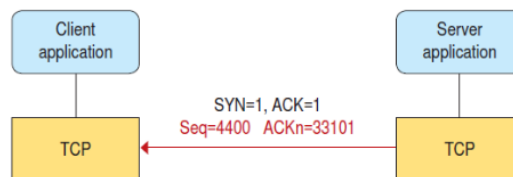
- Il client TCP genera casualmente un **numero di sequenza iniziale** (es. **Seq = 33100**).
- Invia un segmento TCP con:
 - **SYN = 1** (richiesta di sincronizzazione)
 - **ACK = 0**
 - Sequence Number = 33100



4. Risposta del Server (SYN + ACK)

Il server riceve il SYN e risponde con un segmento **SYN/ACK**:

- Conferma di aver ricevuto il SYN del client → **ACK = 1**, Acknowledgment Number = **33101** (33100 + 1)
- Invia il proprio numero di sequenza iniziale (es. **Seq = 4400**)
- Imposta **SYN = 1** (vuole sincronizzare anche il suo flusso)



5. Conferma finale del Client (ACK)

Il client riceve il SYN/ACK e risponde con:

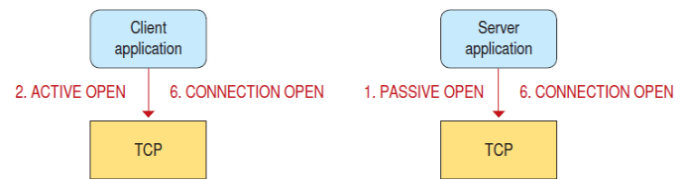
- Acknowledgment Number = **4401** (4400 + 1)
- Sequence Number = **33101**



6. Connessione aperta

A questo punto la connessione è **ufficialmente aperta** in entrambe le direzioni (full-duplex).

Client e Server notificano alle rispettive applicazioni che la connessione è stabilita (**CONNECTION OPEN**).



Riassunto grafico dell'apertura TCP (Three-Way Handshake)

1. **Client** → **Server**: SYN, Seq=33100
2. **Server** → **Client**: SYN, ACK, Seq=4400, Ack=33101
3. **Client** → **Server**: ACK, Seq=33101, Ack=4401

Perché tre messaggi?

- Serve a sincronizzare **entrambi** i numeri di sequenza (uno per ogni direzione).
- Garantisce che entrambe le parti siano pronte e abbiano ricevuto correttamente le informazioni.

La connessione TCP - Chiusura (Four-Way Handshake)

TCP considera la connessione come **due flussi unidirezionali** indipendenti (uno per ogni direzione).

La chiusura può avvenire in due modi:

- **Handshake a 3 vie**: chiusura simultanea (simile all'apertura, ma con flag **FIN** invece di **SYN**).
- **Handshake a 4 vie** (più comune): chiusura una direzione alla volta. Un host invia FIN, riceve ACK, e l'altro host fa lo stesso (4 pacchetti totali).

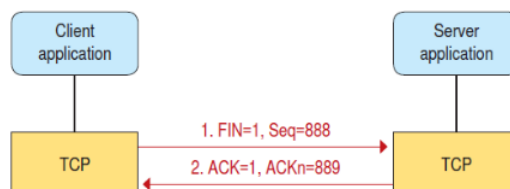
Procedura tipica (4 vie):

1. Il Client decide di chiudere

- Invia un segmento con il flag **FIN = 1** (es. Seq=888)
- Questo segmento può contenere anche gli ultimi dati da trasmettere.

2. Il Server conferma la ricezione

- Risponde immediatamente con un **ACK = 1**
- Ack Number = 889 (888 + 1)

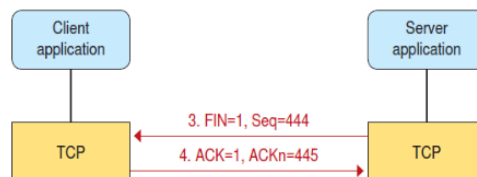


3. Il Server decide di chiudere la sua direzione

- Quando ha finito di inviare i suoi dati, il server invia un segmento con **FIN = 1** (es. Seq=444)

4. Il Client conferma la chiusura definitiva

- Risponde con **ACK = 1**
- Ack Number = 445 (444 + 1)



Riassunto grafico della chiusura TCP (4-way)

1. **Client** → **Server**: FIN=1, Seq=888
2. **Server** → **Client**: ACK=1, Ack=889
3. **Server** → **Client**: FIN=1, Seq=444
4. **Client** → **Server**: ACK=1, Ack=445

Perché TCP usa numeri di sequenza e acknowledgment in apertura e chiusura?

- Per sincronizzare i flussi.
- Per evitare confusione con pacchetti vecchi o duplicati (vecchie connessioni).
- Per garantire che **tutti** i dati siano arrivati prima di chiudere.

La connessione TCP – ritrasmissione

Riscontro positivo con ritrasmissione (Positive Acknowledgment with Retransmission)

Ogni volta che il ricevitore riceve correttamente un segmento, invia un **ACK** (acknowledgment) al mittente per confermare la ricezione.

Meccanismo del timer

Il mittente, quando trasmette un segmento:

- Ne conserva una copia.
- Avvia un **timer**.
- Se il timer scade senza ricevere l'ACK → **ritrasmette** il segmento.

Problema dei riscontri duplicati

Può capitare che la rete duplichi un pacchetto, per evitare confusione, ogni segmento viene **numerato sequenzialmente**, e l'ACK contiene il numero sequenziale del pacchetto che vuole riscontrare.

Problema del riscontro positivo semplice

Se il mittente deve aspettare l'ACK di **ogni** segmento prima di inviare il successivo, la rete viene sottoutilizzata (i pacchetti viaggiano solo in una direzione mentre si aspetta la risposta), quindi si hanno dei rallentamenti.

Soluzione: Sliding Window (finestra scorrevole)

Lo **Sliding Window** permette di inviare fino a **N segmenti** (dimensione della finestra) senza aspettare ogni singolo ACK.

In questo modo la rete rimane utilizzata anche durante i tempi di attesa degli ACK.

Funzionamento dello sliding window

- Un pacchetto è **non riscontrato** se non ha ancora ricevuto l'ACK.
- La finestra del trasmettitore inizia dal primo pacchetto non riscontrato.
- Tutti i pacchetti **dentro** la finestra possono essere trasmessi.
- Quando arriva l'ACK per il primo pacchetto della finestra la finestra **scorre** in avanti di un pacchetto, permettendo di inviare un nuovo pacchetto (oltre a quelli da finire ad inviare già presenti nella finestra).

Prestazioni della sliding window

- Le finestre continuano a scorrere finché si ricevono riscontri, permettendo di trasmettere i nuovi pacchetti che entrano nella finestra.
- Le prestazioni dei protocolli a finestra scorrevole dipendono dalla dimensione della finestra e dalla velocità della rete.
- Se la finestra ha dimensione **1** → si torna al riscontro positivo semplice.
- All'aumentare della dimensione della finestra → diminuisce il tempo di inutilizzo della rete.
- Una dimensione opportuna della finestra permette di **saturare** la rete senza congestionarla.
- Per ogni segmento trasmesso parte comunque un timer per la ritrasmissione.

Sliding window in TCP (bidirezionale e byte-oriented)

Il meccanismo vale in **entrambe** le direzioni (ogni host è contemporaneamente mittente e ricevitore).

Il mittente usa 3 numeri:

1. inizio finestra → primo byte non confermato
2. fine finestra → limite da non superare
3. prossimo byte da inviare

I riscontri del TCP fanno riferimento alla posizione di **byte nello stream** e non ai segmenti o ai datagrammi (non di pacchetto):

- Il sequence number indica la posizione del byte nello stream.
- L'ACK indica il prossimo byte atteso (riscontri cumulativi).
- Il ricevitore riordina i byte che arrivano disordinati dai datagrammi IP e individua eventuali buchi.

Vantaggi e svantaggi dei riscontri cumulativi

Vantaggi:

- Facili da generare.
- Se si perde un ACK, basta che arrivi un ACK successivo per confermare tutti i byte precedenti.

Svantaggio:

- Se manca un byte, il mittente non sa se si tratta di un piccolo "buco" o di una perdita più grande → ritrasmette **tutto** a partire dal byte mancante.

Con questo meccanismo TCP garantisce un trasferimento **affidabile**, ordinato e controllato della connessione end-to-end.

